

Egyszerű elektronikus telefonkönyv programozói dokumentáció

Bevezetés

Az "Egyszerű Elektronikus Telefonkönyv" egy egyszerű, konzolalapú alkalmazás, amely lehetővé teszi telefonkönyv-rekordok kezelését. A programban minden rekord egy telefonszámhoz kapcsolódó személy adatait tárolja, mint például a nevét (vezetéknév és keresztnév), címét, foglalkozását, életkorát és telefonszámát.

A felhasználónak lehetősége van új rekordok létrehozására, a régiek módosítására, törlésére is. A felhasználónak még van lehetősége rekordok keresésére (név, foglalkozás, telefonszám alapján), illetve a megfelelő rekord kiválasztásával vCard megvalósítására is.

Adatszerkezete: A láncolt lista egy dinamikus adatszerkezet, amely lehetővé teszi a rekordok dinamikus tárolását és kezelését. Minden elem (rekord) tartalmazza a telefonkönyv adatait és egy mutatót (pointert), amely a következő elemre mutat. Ez megkönnyíti az elemek hozzáadását és eltávolítását. A láncolt lista előnye, hogy dinamikusan tud nőni, és nem szükséges előre meghatározni a méretét.

Telefonkönyv struktúra:

- char *v_nev, *k_nev: Vezeték- és keresztnév karaktertömbök.
- char *nev: Teljes név karaktertömb.
- char *cim: Cím karaktertömb.
- char *foglalkozas: Foglalkozás karaktertömb.
- int *kor: Életkor egész szám.
- char *tel: Telefonszám karaktertömb.
- struct tel *kov: Következő elemre mutató pointer.

```
typedef struct tel {  
    char *v_nev;  
    char *k_nev;  
    char *nev;  
    char *cim;  
    char *foglalkozas;  
    int kor;  
    char *tel;  
    struct tel *kov;  
} tel;
```

Főprogram (main)

Működési folyamat:

1. Induláskor:

- A program **ASCII art** formájában megjelenít egy telefonkönyvet szimbolizáló grafikát.
- Elmagyarázza a felhasználónak a működést, majd várja, hogy az **"Enter"** billentyű lenyomásával folytassa.

2. Adatok betöltése:

- A program megnyit egy **tel_.txt nevű** fájlt olvasásra, amely tartalmazza a névjegyeket.
- A fájl tartalmából **láncolt lista** jön létre, ahol minden rekordot egy tel típusú csomópontot reprezentál.

3. Főmenü:

- A választásokat egy **switch** szerkezet kezeli, és minden opcióhoz külön funkció tartozik (pl. elso_menu az új rekord létrehozásához).

4. Kilépéskor:

- A program az aktuális listát visszamenti a fájlba.
- A láncolt listát felszabadítja a memóriából, hogy elkerülje a memória-szivárgást.

Lényeges program utasítás: **Láncolt lista**

```

fp = fopen("tel_.txt", "r");           Megnyitja a fájlt olvasási módban ("r").

tel *fej = NULL, *akt = NULL;         Láncolt lista kezdeti állapota

while (true) {                         Addig olvassa az adatokat amíg van elérhető adat
    tel* uj_tel = tel_letrehozás(); Lefoglaljuk a memóriát vagyis létrehozzuk
    int eredmény = fscanf(fp, "%s %s %s %s %d %s", uj_tel->v_nev, uj_tel->k_nev, uj_tel->cim,
uj_tel->foglalkozas, &uj_tel->kor, uj_tel->tel);
    if (eredmény != 6) {
        tel_lista_felszabadítás(uj_tel); A fájl végére érve, nem tudjuk beolvasni az adatokat
                                           felszabadítjuk

        break;
    }
    else {
        strcpy(uj_tel->nev, uj_tel->v_nev);
        strcat(uj_tel->nev, " ");
        strcat(uj_tel->nev, uj_tel->k_nev);
    }

    // Hozzáadjuk az aktot a láncolt listához
    if (fej == NULL) {
        fej = akt = uj_tel;
    }
    else {
        akt->kov = uj_tel;
        akt = akt->kov;
    }
}
}
fclose(fp);                           Bezárjuk a fájlt

```

Összefűzés

case 6: ->Kilépés

```

//Kilépés után a fájlba írjuk az adatokat
fp = fopen("tel_.txt", "w");           Megnyitja a fájlt olvasási módban ("w").
tel *akt = fej;
while (akt->kov != NULL) {
    fprintf(fp, "%s %s %s %d %s\n", akt->nev, akt->cim, akt->foglalkozas, akt->kor, akt-
>tel);
    akt = akt->kov;
}
fprintf(fp, "%s %s %s %d %s\n", akt->nev, akt->cim, akt->foglalkozas, akt->kor, akt-
>tel);
fclose(fp);                           Bezárjuk a fájlt
//Felszabadítjuk a memóriát
tel_lista_felszabadítás(fej);

```

Rekord létrehozása (1.menü)

A felhasználó létre tudja hozni a számára megfelelő rekordot a rekord létrehozása menüben.

A program bekéri a szükséges adatokat:

- Vezetéknév
- Keresztnév
- Cím
- Foglalkozás
- Kor
- Telefonszám

Az első menühöz tartozó függvény: void elso_menu(tel* fej), aktiválható a főmenü „1” gombjának megnyomásával.

void elso_menu(tel* fej)

Leírás: Az **elso_menu** függvény egy új rekord létrehozására szolgál a telefonkönyv láncolt listájában. A felhasználótól adatokat kér be, és ellenőrzi azok helyességét. Ha az adatok megfelelőek, a rekordot hozzáfűzi a lista végéhez.

A függvény paraméterként megkapja a lista első elemére mutató pointert (**fej**).

A rekord létrehozásának lépései:

1. **Rekord inicializálása:** Az új rekordot a **tel_letrehozas()** függvény hozza létre.
2. **Adatok bekérése:** A program bekéri a *vezetéknév*, *keresztnév*, *cím*, *foglalkozás*, *kor* és *telefonszám* értékeket. Az adatok helyességét ellenőrzi (pl. érvényes számjegyek a telefonszámhoz).
3. **Adatok tárolása:** Az új rekord adatmezői feltöltésre kerülnek a bekért adatokkal. A vezetéknév és keresztnév összefűzésre kerül egy teljes névmezőbe.
4. **Rekord hozzáfűzése:** Az új rekord a láncolt lista végére kerül csatolásra.
5. **Hibakezelés:** Ha a felhasználó hibás adatot ad meg (pl. negatív életkor), a program hibaüzenetet ír ki, és új adatot kér. A folyamat megszakítható, ha a vezetéknévként „0” kerül megadásra.
6. **Használt segédfüggvények:**
 - **tel_letrehozas():** Létrehozza az új telefonkönyv rekordot.
 - **tel_lista_felszabaditas():** A rekord létrehozásának megszakításakor felszabadítja a memóriát.
 - **system("cls"):** A képernyő törlésére szolgál, hogy tiszta állapotot biztosítson a felhasználónak.
 - **strcpy(), strcat():** Az új rekordok adatainak másolására és összefűzésére szolgál.

Rekord módosítás (2.menü)

A felhasználó a rekord módosításnál láthatja az adott rekordokat a konzolon és azok alapján ki tudja választani a számára megfelelő rekord módosítását, amit a rekordsorszáma beírásával meg is tudd valósítani. **A második menühöz tartozó függvény:** void masodik_menu(tel* fej), aktiválható a főmenü „2” gombjával.

void masodik_menu(tel* fej)

Leírás: A masodik_menu függvény lehetőséget biztosít arra, hogy a felhasználó módosítson egy meglévő rekordot a telefonkönyvben. A felhasználó kiválaszthatja a módosítani kívánt rekordot, majd új adatokat adhat meg, amelyek felülírják a meglévő értékeket.

A függvény paraméterként megkapja a lista első elemére mutató pointert (**fej**).

A rekord módosításának lépései:

1. **Rekordok kiírása:** A program először kiírja az összes rekordot, és minden rekordhoz sorszámot rendel.
2. **Felhasználói választás:** A felhasználó kiválasztja a módosítani kívánt rekord sorszámát.
3. **Új adatok bekérése:** A program kéri az új adatokat (név, cím, foglalkozás, kor, telefonszám).
4. **Adatok ellenőrzése:** A program ellenőrzi, hogy az új adatok érvényesek-e (pl. a telefonszám helyes formátumú, a kor 0 és 100 közötti érték).
5. **Rekord frissítése:** Ha az adatok helyesek, a rekord frissül. Ha a felhasználó 0-t választ, a módosítás megszakad, és visszatér a főmenübe.
6. **Hibakezelés:** Ha a felhasználó érvénytelen adatokat ad meg (pl. a telefonszám nem 11 számjegyű), a program újra kéri a helyes adatokat. Ha a felhasználó 0-t választ, a módosítás megszakad, és visszatér a főmenübe.

7. Használt segédfüggvények:

- **void listazas(tel *akt, int rekord_szam):** Kiírja az összes telefonkönyv rekordot és azok sorszámait.
- **system("cls"):** A képernyő törlésére szolgál, hogy tiszta állapotot biztosítson a felhasználónak.
- **getchar():** A felesleges karakterek eltávolítására szolgál, amelyek a bemeneti pufferben maradnak a **scanf** használata után. Ez biztosítja, hogy a felhasználó utáni karakterek ne zavarják a program működését.
- **strcpy(), strcat():** Az új rekordok adatainak másolására és összefűzésére szolgál.
- **strcmp():** Összehasonlítja a két karakterláncot. A program a felhasználó által megadott adatokat hasonlítja össze (pl. ha a felhasználó "0"-t ír be, akkor nem módosítja az adatot). Ha a két karakterlánc megegyezik, a függvény 0 értéket ad vissza.

Rekord törlése (3.menü)

A rekord törlése menünél is megjelennek a rekordok a konzolon és szabadon kiválaszthatja a felhasználó melyik rekordot szeretné törölni, amit szintén a rekordsorszáma alapján meg is tud tenni. **A harmadik menühöz tartozó függvény:** void harmadik_menu(tel* fej), aktiválható a főmenü „3” gombjával.

void harmadik_menu(tel* fej)

Leírás: A függvény a telefonkönyv rekordjainak törlésére szolgál. A felhasználó kiválaszthatja, hogy melyik rekordot szeretné törölni, és a program törli azt a láncolt listából.

A függvény paraméterként megkapja a lista első elemére mutató pointert (**fej**).

A rekord törlésének lépései:

1. **Rekordok kiírása:** A program először kiírja az összes rekordot, és minden rekordhoz sorszámot rendel.
2. **Rekord törlés:** A felhasználó bevitelét várja, hogy kiválassza a törölni kívánt rekord sorszáma. A program biztosítja, hogy csak a létező rekordok sorszáma választható, a bemenetet pedig folyamatosan ellenőrzi. Ha a felhasználó 0-t ír be a rekord törlésére, a program az **system("cls")** hívás után kilép a menüből, és nem történik törlés.
3. **Rekord keresés és törlés:** A kiválasztott rekordot a program megkeresi, majd eltávolítja a láncolt listából. Ha az első rekordot kell törölni, akkor a fej pointert frissíti. Ha nem az első rekordot töröljük, az előző rekord mutatóját frissíti, hogy átugorja a törölt elemet.
4. **Memória felszabadítás:** A törölt rekord memóriáját a **tel_felszabaditas()** segédfüggvény hívásával felszabadítja a program.
5. **Használt segédfüggvények:**
 - **void listazas(tel *akt, int rekord_szam):** Kiírja az összes telefonkönyv rekordot és azok sorszámaikat.
 - **system("cls"):** A képernyő törlésére szolgál, hogy tiszta állapotot biztosítson a felhasználónak.
 - **getchar():** A felesleges karakterek eltávolítására szolgál, amelyek a bemeneti pufferben maradnak a **scanf** használata után. Ez biztosítja, hogy a felhasználó utáni karakterek ne zavarják a program működését.
 - **void tel_felszabaditas(tel* rekord):** A kiválasztott rekord memóriáját felszabadítja, miután az eltávolításra került a láncolt listából.

Keresés (4.menü)

A keresés menüben különböző paraméterek (név, foglalkozás, telefonszám) alapján kereshetünk a telefonkönyv rekordjai között. Az almenüben választhatunk, hogy az összes rekordot listázzuk, vagy csak a kívánt keresési kritériumnak megfelelőeket.

A negyedik menühöz tartozó függvény: void negyedik_menu(tel* fej), aktiválható a főmenü „4” gombjával.

void negyedik_menu(tel* fej)

Leírás: A **negyedik_menu** függvény a telefonkönyv keresési funkcióit biztosítja a felhasználónak. A felhasználó választási lehetőségeket kap a rekordok megjelenítésére és keresésére különböző paraméterek szerint (pl. név, foglalkozás, telefonszám). A keresési eredmények alapján a felhasználó megtekintheti a telefonkönyvben szereplő rekordokat.

A függvény paraméterként megkapja a lista első elemére mutató pointert (**fej**).

A rekord keresésének lépései:

- Keresési menü megjelenítése:** A függvény megjeleníti a felhasználó számára a keresési opciókat, amelyeket sorszámozva talál. A felhasználó választhat, hogy az alábbiak közül melyik műveletet szeretné végrehajtani:
 - Listázza az összes rekordot.
 - Keresés név alapján.
 - Keresés foglalkozás alapján.
 - Keresés telefonszám alapján.
 - Keresés név és telefonszám alapján.
 - Kilépés a menüből, vissza a főmenübe.
- Bemenet ellenőrzése:** A felhasználó választása beolvasásra kerül **scanf()** segítségével. Ha a felhasználó nem megfelelő értéket ad meg, a program hibaüzenetet jelenít meg, és újra kéri a bemenetet.
- Kiválasztott keresési művelet végrehajtása:**
 - A **switch** szerkezet segítségével a program a megfelelő keresési műveletet hajtja végre a felhasználó választása alapján.
 - 1 - Listázás:** A rekordok listázása történik a **listazas()** függvény használatával, amely minden egyes rekordot kiír a képernyőre.
 - 2 - Név alapú keresés:** A **nev_alapjan_kereses()** függvény hívódik, amely a rekordokat név szerint keresi meg.
 - 3 - Foglalkozás alapú keresés:** A **foglalkozas_alapjan_kereses()** függvény hívódik, amely a rekordokat foglalkozás szerint keresi.
 - 4 - Telefonszám alapú keresés:** A **telefonszam_alapjan_kereses()** függvény hívódik, amely a rekordokat telefonszám szerint keresi.
 - 5 - Név és telefonszám alapú keresés:** A **nev_es_telefonszam_alapjan_kereses()** függvény hívódik, amely név és telefonszám együttes szűrésével keres.
 - 6 - Kilépés:** A program kilép a keresési menüből és visszatér a főmenübe.
- Használt segédfüggvények:**

- **void listazas(tel *akt, int rekord_szam):** Kiírja az összes telefonkönyv rekordot és azok sorszámaikat.
- **system("cls"):** A képernyő törlésére szolgál, hogy tiszta állapotot biztosítson a felhasználónak.
- **getchar():** A felesleges karakterek eltávolítására szolgál, amelyek a bemeneti pufferben maradnak a **scanf** használata után. Ez biztosítja, hogy a felhasználó utáni karakterek ne zavarják a program működését.



Itt találunk egy almenüt, amit 6 részre osztottam:

- **Az első az a listázás, amiben a rekordokat ki tudjuk listázni**

Ezt a függvényt az „1” beütésével lehet elérni az almenüben.

void listazas(tel* akt, int rekord_szam)

Feladata: Ez a függvény az összes rekord listázásához használható. Az aktuális adatokat rekordszámmal megjelölve sorolja fel, hogy könnyen áttekinthető legyen a telefonkönyv teljes tartalma.

- A függvény paraméterként megkapja a lista első elemére mutató pointert (**fej**).
- **int rekord_szam:** A rekord sorszáma, amelyet a listázás során kiírunk a táblázat első oszlopába.

Függvény működése:

1. Fejléc kiírása:

- Az első rekord kiírásakor (**amikor rekord_szam == 1**) a függvény kiírja a táblázat fejlécét, amely tartalmazza a rekordokhoz tartozó mezőneveket, mint például **"Rekord", "Név", "Cím", "Foglalkozás", "Kor",** és **"Telefonszám"**.
- Az oszlopok közötti elválasztásokat **printf()** hívások biztosítják, és egy vonalat is rajzol a fejléc alatt, hogy vizuálisan elkülönítse a rekordokat.

2. Rekordok kiírása:

- A rekord adatainak kiírása történik a következő módon:
 - **rekord_szam:** A rekord sorszáma (az első rekordtól kezdve).
 - **akt->nev:** A személy neve.
 - **akt->cim:** A személy címe.
 - **akt->foglalkozas:** A személy foglalkozása.
 - **akt->kor:** A személy kora.
 - **akt->tel:** A személy telefonszáma.
- Mindezek a formázott szövegekkel a táblázat megfelelő oszlopaiba kerülnek kiírásra.

- **A második az a név alapján keresés**

Ezt a függvényt az „2” beütésével lehet elérni az almenüben.

void nev_alapjan_kereses(tel *akt)

Feladata: Ez a függvény a név szerinti keresés végrehajtására szolgál. Beviteli mezőt biztosít, ahol a felhasználó megadhat egy nevet vagy névrészletet, és a függvény csak azokat rekordokat listázza, amelyek megfelelnek a keresési feltételnek.

A függvény paraméterként megkapja az aktuális pointert (**akt**).

A függvény működése:

1. Felhasználói bemenet:

- A függvény először bekéri a felhasználótól a keresett nevet, amelyet a **keresett_nev** karaktertömbben tárol. A bemenetet a **scanf** segítségével olvassa be.

2. Táblázat fejléce:

- Miután megkaptuk a keresett nevet, a függvény kiír egy táblázatot, amely tartalmazza az egyes rekordok adatait (név, cím, foglalkozás, kor, telefonszám).

3. Rekordok összehasonlítása:

- Ezután a függvény végig iterál a telefonkönyv rekordjain a **while (akt != NULL)** ciklussal. Minden rekordot összehasonlít a keresett névvel a **wildcardkeres** függvény segítségével. Ha a rekord neve egyezik a keresett névvel, akkor kiírja a rekordot.

4. Keresés eredménye:

- Ha bármelyik rekord neve megegyezik a keresett névvel, akkor az adott rekord adatai ki lesznek írva.
- Ha nincs találat, akkor a függvény kiírja a **"Nincs találat!"** üzenetet.

bool osszehas(char egyik[], char masik[], int darab)

- Az osszehas függvény két karaktertömb paramétert kap (egyik és masik), valamint egy darab számot, amely azt jelzi, hány karaktert kell összehasonlítani.
- A függvény egy for ciklussal megy végig a karaktertömbökön, és összehasonlítja az egyes karaktereket az azonos indexű pozíciókon.
- Ha talál olyan karakterpárt, amelyek nem egyeznek meg, azonnal false értékkel tér vissza.
- Ha a ciklus végéig minden karakter egyezik, akkor true értékkel tér vissza.

```
bool osszehas(char egyik[], char masik[], int darab) {  
    for (int i = 0; i < darab; i++)  
        if (egyik[i] != masik[i]) return false;  
    return true;}  

```

bool wildcardkeres(char keres[], char szo[])

- A wildcardkeres függvény két karaktertömb paramétert kap (keres és szo), és azt vizsgálja, hogy a szo karaktertömb illeszkedik-e a keres által definiált wildcard mintára.
- A függvény egy for ciklussal megy végig a keres karaktertömbön.
- Ha a jelenlegi karakter egy '*' karakter, akkor meghívja az osszehas függvényt az előző karakterek összehasonlítására. Ha az összehasonlítás sikeres, akkor rekurzívan meghívja önmagát a következő karakterekre és a szo tömb maradék részére.
- Ha az osszehas nem teljesül (tehát a minta nem egyezik meg a szóval az adott ponton), akkor azonnal false értékkel tér vissza.
- Ha a ciklus végére ér, akkor a maradék részt (amennyiben van) ugyanezen a módon ellenőrzi, majd összehasonlítja a teljes keres és szo tömböket a strcmp függvény segítségével. Ha egyeznek, akkor true értékkel tér vissza, különben false értékkel.

```
bool wildcardkeres(char keres[], char szo[]) {  
    for (int i = 0; i < strlen(keres); i++) {  
        if (keres[i] == '*') {  
            // Ellenőrizzük, hogy a csillag után következő karakterek megfelelnek-e a 'szo'  
            karakterláncnak  
            if (!osszehas(keres, szo, i))  
                return false;  
            if (keres[i + 1] == '\\0')  
                return true;  
            for (int a = i; a < strlen(szo); a++)  
                // Rekurzívan hívjuk a wildcardkeres függvényt a következő karakterekkel  
                if (wildcardkeres(&keres[i + 1], &szo[a]))  
                    return true;  
        }  
    }  
    if (!strcmp(keres, szo))  
        return true;  
    else  
        return false;}  
}
```

- **A harmadik a foglalkozás alapján keresés**

Ezt a függvényt az „3” beütésével lehet elérni az almenüben.

void foglalkozas_alapjan_kereses(tel *akt)

Feladata: A függvény a rekordokat foglalkozás alapján szűri. A felhasználó beírhatja a keresett foglalkozást, és a függvény csak azokat a rekordokat jeleníti meg, amelyeknél a foglalkozás egyezik a megadott feltétellel.

A függvény paraméterként megkapja az aktuális pointert (**akt**).

A függvény működése:

1. Felhasználói bemenet:

- A függvény először bekéri a felhasználótól a keresett nevet, amelyet a **keresett_foglalkozas** karaktertömbben tárol. A bemenetet a **scanf** segítségével olvassa be.

2. Táblázat fejléce:

- Miután megkaptuk a keresett foglalkozást, a függvény kiír egy táblázatot, amely tartalmazza az egyes rekordok adatait (név, cím, foglalkozás, kor, telefonszám).

3. Rekordok összehasonlítása:

- Ezután a függvény végig iterál a telefonkönyv rekordjain a **while (akt != NULL)** ciklussal. Minden rekordot összehasonlít a keresett foglalkozással az **strcmp(akt->foglalkozas, keresett_foglalkozas)** függvény segítségével. Ha a rekord neve egyezik a keresett foglalkozással, akkor kiírja a rekordot.

4. Keresés eredménye:

- Ha bármelyik rekord foglalkozás megegyezik a keresett foglalkozással, akkor az adott rekord adatai ki lesznek írva.
- Ha nincs találat, akkor a függvény kiírja a **"Nincs találat!"** üzenetet.

A negyedik az a telefonszám alapján keresés:

Ezt a függvényt az „4” beütésével lehet elérni az almenüben.

void telefonszám_alapjan_kereses(tel* akt)

Feladata: Ezzel a függvénnyel telefonszám alapján lehet keresni a rekordok között. Lehetőséget biztosít arra, hogy a felhasználó beírjon egy telefonszámot, és a rendszer csak a hozzá tartozó rekordot vagy rekordokat listázza ki.

A függvény paraméterként megkapja az aktuális pointert (**akt**).

A függvény működése:**1. Felhasználói bemenet:**

- A függvény először bekéri a felhasználótól a keresett telefonszámot, amelyet a **keresett_telefonszam** karaktertömbben tárol. A bemenetet a **scanf** segítségével olvassa be.

2. Táblázat fejléce:

- Miután megkaptuk a keresett telefonszámot, a függvény kiír egy táblázatot, amely tartalmazza az egyes rekordok adatait (név, cím, foglalkozás, kor, telefonszám).

3. Rekordok összehasonlítása:

- Ezután a függvény végig iterál a telefonkönyv rekordjain a **while (akt != NULL)** ciklussal. Minden rekordot összehasonlít a keresett telefonszámmal a **wildcardkeres** függvény segítségével. Ha a rekord neve egyezik a keresett telefonszámmal, akkor kiírja a rekordot.

4. Keresés eredménye:

- Ha bármelyik rekord telefonszáma megegyezik a keresett telefonszámmal, akkor az adott rekord adatai ki lesznek írva.
- Ha nincs találat, akkor a függvény kiírja a **"Nincs találat!"** üzenetet.

- **Az ötödik az a név és telefonszám alapján keresés:**

Ezt a függvényt az „4” beütésével lehet elérni az almenüben.

void nev_es_telefonszam_alapjan_kereses(tel* akt)

Feladata: Ezzel a függvénnyel név és telefonszám együttes megadásával lehet keresni. A felhasználó megadhat egy nevet és hozzá egy telefonszámot, és a függvény csak azokat a rekordokat listázza, amelyek mindkét feltételnek megfelelnek.

A függvény paraméterként megkapja az aktuális pointert (**akt**).

A függvény működése:

1. Felhasználói bemenet:

- A függvény először bekéri a felhasználótól a keresett nevet és a keresett telefonszámot, amelyeket a **keresett_nev** és a **keresett_telefonszam** karaktertömbben tárol. A bemenetet a `scanf` segítségével olvassa be. A név bemenetet a `scanf(" %[^\n]", keresett_nev)` olvassa be, amely lehetővé teszi a szóközök kezelését a névben.

2. Táblázat fejléce:

- Miután megkaptuk a keresett telefonszámot, a függvény kiír egy táblázatot, amely tartalmazza az egyes rekordok adatait (név, cím, foglalkozás, kor, telefonszám).

3. Rekordok összehasonlítása:

- Minden rekordot egyesével vizsgál meg a láncolt listában a **while (akt != NULL)** ciklussal.
- A **wildcardkeres** függvénnyel ellenőrzi, hogy:
 - A név (**akt->nev**) részben vagy egészében egyezik-e a keresett névvel.
 - A telefonszám (**akt->tel**) részben vagy egészében egyezik-e a keresett telefonszámmal.

4. Keresés eredménye:

- Ha a név és a telefonszám **is megfelel**, az aktuális rekord adatait kiírja a táblázatba.
- Ha nincs találat, akkor a függvény kiírja a "Nincs találat!" üzenetet.

- **A hatodik az a vissza a főmenübe, ami visszavisz a kezdőlapra.**

Ezt a függvényt az „6” beütésével lehet elérni az almenüben.

vCard (5.menü)

A felhasználó megadja az adott rekordnak a számát, amit láthat a konzolon és az alapján a program megcsinálja az adott rekordnak a vCard-ját. A rekordsorszáma alapján tudja kiválasztani a neki megfelelő rekord vCard-ját.

Az ötödik menühöz tartozó függvény: void otodik_menu(tel* fej), aktiválható a főmenü „5” gombjával.

void otodik_menu(tel* fej)

Leírása: Az **otodik_menu** függvény lehetőséget biztosít a felhasználónak arra, hogy a telefonkönyv rekordjai közül válasszon, és a kiválasztott rekord alapján egy vCard fájlt generáljon. A vCard egy szabványos formátum névjegyek tárolására, amely kompatibilis különböző alkalmazásokkal.

A függvény paraméterként megkapja a lista első elemére mutató pointert (**fej**).

A rekord vCardjának lépései:

- 1. Rekordok kiírása:** A program megjeleníti az összes telefonkönyvi rekordot a konzolon, sorszámmal ellátva.
- 2. Felhasználói választás:** A felhasználó megadhatja annak a rekordnak a sorszámát, amelyből vCard fájlt szeretne készíteni. A választás előtt a program ellenőrzi a megadott sorszám helyességét. Kilépési lehetőség: Ha a felhasználó 0-t ír be, a program kilép az ötödik menüből, és visszatér a főmenübe.
- 3. vCard generálás:** A program a kiválasztott rekord adatai alapján elkészíti a vCard fájlt egy segédfüggvény (**vCard_keszit**) segítségével.
- 4. Hibakezelés:**
 - Ha a felhasználó érvénytelen sorszámot ad meg (pl. negatív vagy a rekordok számán kívüli értéket), a program újra kéri a helyes adatot.
 - Ha a felhasználó nem egész számot ír be, a bemeneti hibát kezeli, és újra kéri a választást.
- 5. Használt segédfüggvények:**
 - **void listazas(tel *akt, int rekord_szam):** Kiírja az összes telefonkönyv rekordot és azok sorszámaikat.
 - **system("cls"):** A képernyő törlésére szolgál, hogy tiszta állapotot biztosítson a felhasználónak.
 - **getchar():** A felesleges karakterek eltávolítására szolgál, amelyek a bemeneti pufferben maradnak a **scanf** használata után. Ez biztosítja, hogy a felhasználó utáni karakterek ne zavarják a program működését.
 - **vCard_keszit(tel *akt):** A kiválasztott rekord alapján vCard fájlt készít.

void vCard_keszit(tel* akt)

Feladata: Ez a függvény egy vCard formátumú virtuális névjegykártyát hoz létre és ment el egy fájlban a megadott 'tel' típusú struktúra alapján.

A függvény paraméterként megkapja az aktuális pointert (**akt**).

A függvény működése:

1. Fájlnev generálása:

- A v_nev és k_nev mezők alapján hozza létre a fájl nevét.
- Példa: ha v_nev = "Nagy" és k_nev = "Istvan", akkor a fájl neve Nagy_Istvan.vcf lesz.

2. vCard adatok írása:

- A szabványos vCard 3.0 mezők kerülnek bele:
 - FN: Teljes név.
 - ORG: Foglalkozás.
 - TEL: Telefonszám.

3. ASCII grafika megjelenítése:

- A terminálon ASCII art jelenik meg, amely tartalmazza a névjegykártya vizuális reprezentációját, és megjeleníti a név, foglalkozás, telefonszám adatokat.

A **vCard_keszit** függvény futása után az elkészült .vcf fájl a program aktuális munkakönyvtárában található. Ez a könyvtár az, ahol a programot elindították. A fájl nevét a függvény automatikusan generálja a v_nev (vezetéknév) és k_nev (keresztnev) mezők alapján.

```
FILE *vCardFile;
char vCardFilename[50];

// Vezetéknév és keresztnév összefűzése
sprintf(vCardFilename, "%s_%s.vcf", akt->v_nev, akt->k_nev);

// vCard fájl megnyitása
vCardFile = fopen(vCardFilename, "w");
if (vCardFile == NULL) {
    perror("Nem sikerült megnyitni a vCard fajt");
}
// vCard adatok írása a fájlba
fprintf(vCardFile, "BEGIN:VCARD\n");
fprintf(vCardFile, "VERSION:3.0\n");
fprintf(vCardFile, "FN:%s\n", akt->nev);
fprintf(vCardFile, "ORG:%s\n", akt->foglalkozas);
fprintf(vCardFile, "TEL:%s\n", akt->tel);
fprintf(vCardFile, "END:VCARD\n");

// Fájl bezárása
fclose(vCardFile);
```

Kilépés (6.menü)

Feladata: Ha ezt a menüt választjuk akkor teljesen elhagyjuk a főmenüt és kilépünk a programból.

Egyéb függvények:

tel* tel_letrehozas()

Leírás: Létrehoz egy új telefonkönyv elemet, amely lehetőséget biztosít egy új rekord inicializálására. Ez a függvény felelős azért, hogy minden szükséges adatmező megfelelő memóriaterületet kapjon a rekord tárolásához.

```
tel* tel_letrehozas() {
    tel* uj_tel = (tel*)malloc(sizeof(tel));
    if (uj_tel == NULL) {
        perror("Memória foglalási hiba");
        exit(EXIT_FAILURE);
    }

    //Lefoglaljuk a memóriát
    uj_tel->v_nev = (char*)malloc(16 * sizeof(char));
    uj_tel->k_nev = (char*)malloc(16 * sizeof(char));
    uj_tel->nev = (char*)malloc(32 * sizeof(char));
    uj_tel->cim = (char*)malloc(21 * sizeof(char));
    uj_tel->foglalkozas = (char*)malloc(16 * sizeof(char));
    uj_tel->tel = (char*)malloc(12 * sizeof(char));
    uj_tel->kov = NULL;

    return uj_tel;
}
```


void tel_lista_felszabaditas(tel* fej)

Leírás: Felszabadítja a telefonkönyv listáját és az összes kapcsolódó erőforrást. Ez a függvény létfontosságú a program memória-hatékonyságának fenntartásában, különösen akkor, amikor a teljes listát töröljük, vagy a program kilép.

- **fej:** A telefonkönyv lista első elemére mutató pointer.

```
void tel_lista_felszabaditas(tel* fej) {  
  
    //Felszabadítjuk a memóriát  
    if (fej != NULL) {  
        if (fej->kov != NULL) tel_lista_felszabaditas(fej->kov);  
        free(fej->v_nev);  
        free(fej->k_nev);  
        free(fej->nev);  
        free(fej->cim);  
        free(fej->foglalkozas);  
        free(fej->tel);  
        free(fej);  
    }  
}
```

void tel_felszabaditas(tel* rekord)

Leírás: Felszabadítja a megadott telefonkönyv rekord által lefoglalt memóriaterületet. Ez a függvény a törlési folyamatban, a harmadik menüpontban különösen fontos szerepet játszik, mivel gondoskodik arról, hogy a törölni kívánt rekord memóriája felszabaduljon, így elkerülve az esetleges memória-szivárgásokat. Abban különbözik a fenti függvénytől, hogy nem rekurzívan szabadítunk.

#Include

<stdio.h>: Ez a standard input-output fejlécfájl, és számos alapvető be- és kimeneti műveletet tartalmaz, például az printf és scanf függvényeket.

<stdlib.h>: Ez a standard könyvtár része, és az általános célú függvények és makrók deklarációit tartalmazza. Ide tartoznak például a memóriakezelés függvényei, mint a malloc, free, calloc, realloc.

<stdbool.h>: Ebben a fejlécfájlban található a bool típus definiálása és néhány segéd függvény, mint például a true és false makrók.

<string.h>: Ez a standard string manipulációs függvényeket tartalmazza, mint például a strlen, strcpy, strcmp, strstr.

"debugmalloc.h": Ez egy olyan fejlécfájl, amit a programba be kellett építeni és ellenőrizni, hogy a programban memóriaszivárgás vagy „kanari” probléma van.

Modulok

A programot öt modulra bontottam:

main.c: a főprogramom (menük választása itt található meg).

menuk.c: ehhez hozzátartozik egy (menuk.h), itt a (a menüknek a függvényei találhatóak meg) pl: void elso_menu(tel *fej), void negyedik_menu(tel *fej).

lista.c: ehhez hozzátartozik egy (lista.h), itt a (láncolt listának a memóriefoglalása, illetve felszabadítása található meg).pl: tel* tel_letrehozas(); void tel_lista_felszabaditas(tel* fej).

kereses.c: ehhez hozzátartozik egy (kereses.h), itt a (itt a kereséshez szükséges függvények f igyelhetők meg). pl: bool osszehas(char egyik[], char masik[], int darab) , void nev_alapjan_kereses (tel * akt).

egyeb.c: ehhez hozzátartozik egy (egyeb.h). pl: void vCard keszit(tel *akt); void listazas(tel *akt, int rekord_szam).

Források

A programban megjegyzések formájában megtalálhatóak a források is.

```
//Források:
//InfoC //https://infoc.eet.bme.hu/fajlkezeles/
//https://infoc.eet.bme.hu/scanf/
//https://infoc.eet.bme.hu/scanf_hibakezeles/
//https://infoc.eet.bme.hu/debugmalloc/
//https://infoc.eet.bme.hu/megjelenites/ itt nem használom a c-econio-t csak megnéztem ANSI
escape kódokat milyen a piros szín
//https://gist.github.com/RabaDabaDoba/145049536f815903c79944599c6f952a ANSI escape
kódolás
//pl: printf("\033[1;31m"); itt \33 ANSI escape kódja, 1 az félkövér, a 31 a szöveg színe, ami jelen
esetben a piros
//https://docs.fileformat.com/hu/email/vcf/ itt a vCard
//https://en.wikipedia.org/wiki/VCard itt is vCard
//https://github.com/eventable/vobject itt is vCard
//https://www.asciiart.eu/ ->rajzok
```

Megjegyzések

Fontos! A program nem használ ékezeteket! A program a tel_.txt fájlból dolgozik!

A keresésnél (Wildcard) pl Nagy*; N*; *Istvan; megkeresheti a telefonkönyvben Nagy Istvant.

A forráskódban található kommentek magyarázatot adnak a különböző részekről és funkciókról.

A fenti információk alapján a felhasználó könnyen használhatja a programot a személyes telefonkönyve kezeléséhez és adatai kereséséhez. A szükséges funkciók intuitívak, és a program könnyen testre szabható a felhasználó igényeinek megfelelően. Csak lényeges programutasításokat raktam a dokumentációmba.

A **"Módosítás" (//Módosítás)** megjegyzés a program korábbi, tavalyi verziójához képest végrehajtott frissítéseket és javításokat dokumentálja. Ezek a megjegyzések rögzítik az eredeti funkciók fejlesztéseit, az optimalizálásokat, valamint az esetleges bővítéseket, amelyek az alkalmazás stabilitását és hatékonyságát szolgálják.

pl: A törölt rekordokat piros színnel kezeli a program. (Kizárólag ANSI kódolásban)

Hibaüzenetek: „Érvénytelen válasz” „Hibás bemenet!” „Kérek egy egész számot a 1 - n (egy adott tartomány legnagyobb értéke) tartományban” „Hibás bemenet! Kérek egy egész számot 0 és 100 között:” „Hibás telefonszám! Kérek egy 11 jegyű telefonszámot:”

Készítette: Kálmán Patrik

Neptun kód: XG5YQ1

Keltezés: 2024.11.21

Tartalom

Telefonkönyv struktúra:	1
Főprogram (main)	2
Rekord létrehozása (1.menü)	4
void elso_menu(tel* fej)	4
Rekord módosítás (2.menü)	5
void masodik_menu(tel* fej)	5
Rekord törlése (3.menü)	6
void harmadik_menu(tel* fej)	6
Keresés (4.menü)	7
void negyedik_menu(tel* fej)	7
void listazas(tel* akt, int rekord_szam)	8
void nev_alapjan_kereses(tel *akt)	9
bool osszehas(char egyik[], char masik[], int darab)	9
bool wildcardkeres(char keres[], char szo[])	10
void foglalkozas_alapjan_kereses(tel *akt)	11
void telefonszám_alapjan_kereses(tel* akt)	12
void nev_es_telefonszám_alapjan_kereses(tel* akt)	13
vCard (5.menü)	14
void otodik_menu(tel* fej)	14
void vCard_keszit(tel* akt)	15
Kilépés (6.menü)	16
Egyéb függvények:	16
tel* tel_letrehozas()	16
void tel_lista_felszabaditas(tel* fej)	17
void tel_felszabaditas(tel* rekord)	17
#Include	17
Modulok	18
Források	18